一、图书管理系统_ajax

二、教学目标

1、完成图书管理系统的功能模块

2、了解orm应用-枚举和多对多的三种创建方式

3、掌握ajax核心能力

三、图书管理系统

四、出版社

在home.html中添加url

```
<a href="{% url 'publish_list' %}" class="list-group-item">出版社列表</a>
```

4.0.1 列表

book -> urls.py

```
#出版社列表
path('publish/list/', views.publish_list, name='publish_list'),
```

app01 -> views.py

```
#出版社列表
def publish_list(request):
    publish_queryset = models.Publish.objects.all()
    return render(request,'publish_list.html',locals())
```

新建publish_list.html

```
{% extends 'home.html' %}

{% block content %}
    <h1 class="text-center">出版社列表</h1>
    <a href="" class="btn btn-success pull-left">添加出版社</a>

    <table class="table table-hover table-striped">
        <thead>
```

```
            <tr>
                <th>ID</th>
                <th>名称</th>
                <th>地址</th>
                <th>邮箱</th>
                <th>操作</th>
            </tr>
        </thead>
        <tbody>
            {% for publish_query in publish_queryset %}
                <tr>
                    <td>{{ publish_query.pk }}</td>
                    <td>{{ publish_query.name }}</td>
                    <td>{{ publish_query.address }}</td>
                    <td>{{ publish_query.email }}</td>
                    <td>
                        <a href="" class="btn btn-primary btn-xs">编辑</a>
                        <a href="" class="btn btn-danger  btn-xs">删除</a>
                    </td>
                </tr>
            {% endfor %}

        </tbody>

    </table>
{% endblock %}
```

4.0.1 添加

publish_list.html

```
<a href="{% url 'publish_add' %}" class="btn btn-success pull-left">添加出版社</a>
```

book -> urls.py

```
#添加出版社
path('publish/add/',views.publish_add,name='publish_add'),
```

app01 -> views.py

```
#出版社添加
def publish_add(request):
    return render(request, 'publish_add.html')
```

publish_add.html

```
{% extends 'home.html' %}

{% block content %}
    <h2 class="text-center">添加出版社</h2>
    <form action="" method="post">
        <p>出版社名称：<input type="text" name="name" class="form-control"></p>
        <p>地址：<input type="text" name="address" class="form-control"></p>
        <p>邮箱：<input type="email" name="email" class="form-control"></p>

        <input type="submit" class="btn btn-success" value="提交">

    </form>
{% endblock %}
```

点击提交

app01 -> views.py

```
#出版社添加
def publish_add(request):

    if request.method == 'POST':
            #名称
            name = request.POST.get('name')
            #地址
            address = request.POST.get('address')
            #邮箱
            email = request.POST.get('email')

            #添加数据到数据库
            models.Publish.objects.create(name=name,
                                          address=address,
                                          email=email)
            return redirect('publish_list')
  return render(request, 'publish_add.html')
```

4.0.1 编辑

publish_list.html

```
<a href="{% url 'publish_edit' publish_id=publish_query.pk %}" class="btn btn-
primary btn-xs">编辑</a>
```

book -> urls.py

```
#编辑出版社
re_path('publish/edit/(?P<publish_id>\d+)', views.publish_edit,
name='publish_edit'),
```

app01 -> views.py

```
#出版社编辑
def publish_edit(request,publish_id):
    #根据id查询出版社对象
    publish_obj = models.Publish.objects.filter(pk=publish_id).first()
    return  render(request,'publish_edit.html',locals())
```

publish_edit.html

```
{% extends 'home.html' %}

{% block content %}
    <h2 class="text-center">编辑出版社</h2>
    <form action="" method="post">
        <p>出版社名称：<input type="text" value="{{ publish_obj.name }}"
name="name" class="form-control"></p>
        <p>地址：<input type="text" value="{{ publish_obj.address }}"
name="address" class="form-control"></p>
        <p>邮箱：<input type="email" value="{{ publish_obj.email }}" name="email"
class="form-control"></p>

        <input type="submit" class="btn btn-success" value="提交">

    </form>
{% endblock %}
```

点击提交

app01 -> views.py

```
#出版社编辑
def publish_edit(request,publish_id):
    #根据id查询出版社对象
    publish_obj = models.Publish.objects.filter(pk=publish_id).first()

    if request.method == 'POST':
        # 名称
        name = request.POST.get('name')
        # 地址
        address = request.POST.get('address')
        # 邮箱
        email = request.POST.get('email')

        publish_obj.name = name
        publish_obj.address = address
        publish_obj.email = email
        #修改
        publish_obj.save()
        return redirect('publish_list')



    return  render(request,'publish_edit.html',locals())
```

4.0.1 删除

publish_list.html

```
<a href="{% url 'publish_delete' delete_id=publish_query.pk %}" class="btn btn-danger  btn-xs">删除</a>
```

book -> urls.py

```
#删除出版社
re_path('publish/delete/(?P<delete_id>\d+)', views.publish_delete, name='publish_delete'),
```

app01 -> views.py

```
#出版社删除
def publish_delete(request,delete_id):
    models.Publish.objects.filter(pk=delete_id).delete()
    return redirect('publish_list')
```

五、作者

在home.html中添加url

```
<a href="{% url 'author_list' %}" class="list-group-item">作者列表</a>
```

5.0.1 列表

book -> urls.py

```
 #作者列表
path('author/list/', views.author_list, name='author_list'),
```

app01 -> views.py

```
#作者列表
def author_list(request):
    author_queryset = models.Author.objects.all()
    return render(request,'author_list.html',locals())
```

新建author_list.html

```
{% extends 'home.html' %}

{% block content %}
    <h1 class="text-center">作者列表</h1>
    <a href="{% url 'author_add' %}" class="btn btn-success pull-left">添加出版社
</a>

    <table class="table table-hover table-striped">
```

```
        <thead>
            <tr>
                <th>ID</th>
                <th>姓名</th>
                <th>年龄</th>
                <th>联系方式</th>
                <th>地址</th>
                <th>操作</th>
            </tr>
        </thead>
        <tbody>
            {% for author_query in author_queryset %}
                <tr>
                    <td>{{ author_query.pk }}</td>
                    <td>{{ author_query.name }}</td>
                    <td>{{ author_query.age }}</td>
                    <td>{{ author_query.authordetail.phone }}</td>
                    <td>{{ author_query.authordetail.addr }}</td>
                    <td>
                        <a href="" class="btn btn-primary btn-xs">编辑</a>
                        <a href="" class="btn btn-danger  btn-xs">删除</a>
                    </td>
                </tr>
            {% endfor %}

        </tbody>

    </table>
{% endblock %}
```

5.0.1 添加

author_list.html

```
<a href="{% url 'author_add' %}" class="btn btn-success pull-left">添加作者</a>
```

book -> urls.py

```
#添加作者
path('author/add/',views.author_add,name='author_add'),
```

app01 -> views.py

```
#添加作者
def author_add(request):
    return render(request, 'author_add.html')
```

publish_add.html

```
{% extends 'home.html' %}

{% block content %}
    <h2 class="text-center">添加作者</h2>
    <form action="" method="post">
        <p>姓名：<input type="text" name="name" class="form-control"></p>
        <p>年龄：<input type="text" name="age" class="form-control"></p>
        <p>联系方式：<input type="text" name="phone" class="form-control"></p>
        <p>地址：<input type="text" name="addr" class="form-control"></p>
        <input type="submit" class="btn btn-success" value="提交">

    </form>
{% endblock %}
```

点击提交

app01 -> views.py

```
#添加作者
def author_add(request):
    if request.method == 'POST':
        name = request.POST.get('name')
        age = request.POST.get('age')
        phone = request.POST.get('phone')
        addr = request.POST.get('addr')
        #因为要添加两张表要先拿到主表的对象 一对一
        author_obj = models.Author.objects.create(name=name,age=age)
        models.AuthorDetail.objects.create(phone=phone,
                                           addr=addr,
                                           author=author_obj)
        return render(request, 'author_list.html')

    return render(request, 'author_add.html')
```

5.0.1 编辑

author_list.html

```
<a href="{% url 'author_edit' author_id=author_query.pk %}" class="btn btn-
primary btn-xs">编辑</a>
```

book -> urls.py

```
#编辑出版社
re_path('publish/edit/(?P<publish_id>\d+)', views.publish_edit,
name='publish_edit'),
```

app01 -> views.py

```
#编辑作者
def author_edit(request,author_id):
    #根据id查询出作者对象
    author_obj = models.Author.objects.filter(pk=author_id).first()
    return render(request, 'author_edit.html', locals())
```

publish_edit.html

```
{% extends 'home.html' %}

{% block content %}
    <h2 class="text-center">编辑作者</h2>
    <form action="" method="post">
        <p>姓名：<input type="text" name="name" value="{{ author_obj.name }}"
class="form-control"></p>
        <p>年龄：<input type="text" name="age" value="{{ author_obj.age }}"
class="form-control"></p>
        <p>联系方式：<input type="text" value="{{ author_obj.authordetail.phone
}}" name="phone" class="form-control"></p>
        <p>地址：<input type="text" name="addr" value="{{
author_obj.authordetail.addr }}" class="form-control"></p>
        <input type="submit" class="btn btn-success" value="提交">

    </form>
{% endblock %}
```

点击提交

app01 -> views.py

```
#编辑作者
def author_edit(request,author_id):
    #根据id查询出作者对象
    author_obj = models.Author.objects.filter(pk=author_id).first()

    if request.method == 'POST':
        name = request.POST.get('name')
        age = request.POST.get('age')
        phone = request.POST.get('phone')
        addr = request.POST.get('addr')

        author_obj.name=name
        author_obj.age=age
        author_obj.save()
        author_obj.authordetail.phone=phone
        author_obj.authordetail.addr=addr
        author_obj.authordetail.save()
        return redirect('author_list')


    return render(request, 'author_edit.html', locals())
```

5.0.1 删除

author_list.html

```
<a href="{% url 'author_delete' delete_id=author_query.pk %}" class="btn btn-danger  btn-xs">删除</a>
```

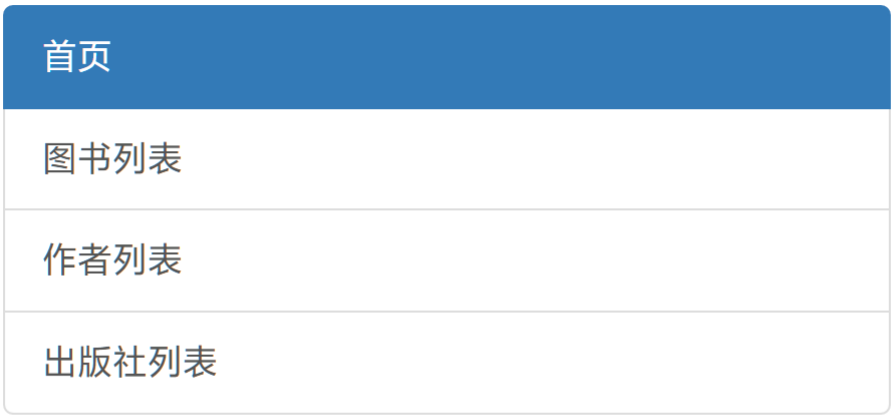
book -> urls.py

```
# 删除作者
re_path('author/delete/(?P<delete_id>\d+)', views.author_delete, name='author_delete'),
```

app01 -> views.py

```
#删除
def author_delete(request,delete_id):
    models.Author.objects.filter(pk=delete_id).delete()
    return redirect('author_list')
```

优化:



发现这个激活一直在首页 我们可以根据用户点击哪个就激活哪个

给首页添加别名

```
#首页
re_path('^$',views.home,name='home')
```

完整优化

```
<!--左边三份 -->
            <div class="col-md-3">
               <div class="list-group">
                  {% url 'home' as home_url %}
```

```
                    <a href="{{ home_url }}" class="list-group-item {% if
request.path == home_url %}active{% endif %}">首页</a>

                    {% url 'book_list' as book_list_url %}
                    <a href="{{ book_list_url }}" class="list-group-item {% if
request.path == book_list_url %}active{% endif %}">图书列表</a>

                    {% url 'author_list' as author_list_url %}
                    <a href="{{ author_list_url }}" class="list-group-item {% if
request.path == author_list_url %}active{% endif %}">作者列表</a>

                    {% url 'publish_list' as publish_list_url %}
                    <a href="{{ publish_list_url }}" class="list-group-item {% if
request.path == publish_list_url %}active{% endif %}">出版社列表</a>
                </div>
            </div>
```

六、补充-orm知识点(了解)

6.0.1 choices 枚举

models.py

用于演示

```
class User(models.Model):
    name = models.CharField(max_length=32)
    age = models.IntegerField()
    #以元组套与元组的形式列举出所有的可能性
    gender_choices=(
        (1,'男'),
        (2, '女'),
        (3, '保密'),
    )
    # 内层元组的第一个值是存于数据库的值，第二个值是对应的值
    gender = models.IntegerField(choices=gender_choices)
```

迁移数据库

```
python manage.py makemigrations
```

执行数据库

```
 python manage.py migrate
```

添加初始数据

```sql
INSERT INTO `django_book`.`app01_user`(`id`, `name`, `age`, `gender`) VALUES (1, 'aaa', 23, 1);
INSERT INTO `django_book`.`app01_user`(`id`, `name`, `age`, `gender`) VALUES (2, 'bbb', 33, 2);
INSERT INTO `django_book`.`app01_user`(`id`, `name`, `age`, `gender`) VALUES (3, 'ccc', 88, 3);
INSERT INTO `django_book`.`app01_user`(`id`, `name`, `age`, `gender`) VALUES (4, 'ddd', 12, 4);
```

test.py

```python
from django.test import TestCase

import os
# Create your tests here.


#1.如果数据库存放的数据不在元组枚举的范围内?
#2.怎么样获取到对应的值


if __name__ == '__main__':
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'book.settings')
    import django
    django.setup()

    from app01 import models

    res = models.User.objects.filter(pk=1).first()
    # 直接对象、属性会返回表存放数字本身
    print(res.gender)
    #获取对应数据,固定写法:get_字段名_display()
    print(res.get_gender_display())

    #如果是不存在的呢?
    res = models.User.objects.filter(pk=4).first()
    # 直接对象、属性会返回表存放数字本身
    print(res.gender)
    # 获取对应数据,固定写法:get_字段名_display()

    #如果存放的值不在元组枚举的范围内，会直接返回存放的值
    print(res.get_gender_display())
```

6.0.1 多对多的三种创建方式

- 全自动创建

```
class Book(models.Model):
    name = ....
    authors = models.ManyToManyField(to='Author')
class Author(models.Model):
    name = ....
```

这种方法会自动创建第三张关系表,而且可以利用 orm 提供操作第三张关系表的快捷方法.

缺点：不能在第三张表中扩展字段

- 纯手动

```
class Book(models.Model):
    name = ....
class Author(models.Model):
    name = ....
class BookToAuthor(models.Model):
    book=models.ForeignKey(to='book',on_de....)
    author=models.ForeignKey(to='author',on_de....)
```

这样方法创建的多对多的话是可以继续其他字段

虽然方便扩展第三张表中的字段

缺点:代码都得自己写,并且不能使用 orm 提供的简单操作

- 半自动

```
class Book(models.Model):
    name = ....
    authors =
models.ManyToManyField(to='Author',through='BookToAuthor',through_field=
('book','author'))
class Author(models.Model):
    name = ....
class BookToAuthor(models.Model):
    book=models.ForeignKey(to='book',on_de....)
    author=models.ForeignKey(to='author',on_de....)
```

> 手动创建3张表.然后在 Book 类中或者 Author中创建 models.ManyToManyField().传入的参数
>
> to= 关联表
>
> through= 第三张关系表
>
> through_fields('book', 'author')
>
> models.ManyToManyField() 这个属性写在哪个类下面,就将哪个类对应的外键字段写前面

可以使用orm提供正反向查询

缺点：不能使用 add,remove,set,clear 这四个方法.

七、ajax 请求

概念:

异步通信技术，无需刷新页面即可与服务器交换数据并更新部分网页内容

核心特点

- 异步通信：无需刷新整个页面
- 部分更新：只更新需要变化的区域
- 数据交换：常用 JSON/XML 格式
- 用户无感：后台静默处理数据

后端发起请求的方式:

- 浏览器地址栏直接输入 url 回车,get
- 请求a标签的 href属性,get
- 请求form 表单,可以发送 get/post
- 请求ajax 请求,可以发送 get/post 请求

7.0.1 案例演示

有三个input 框,在第一个和第二个input 框中分别输入两个数字

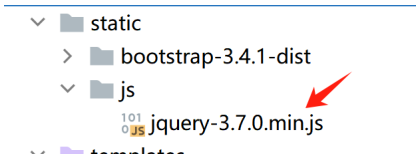
点击'='按钮,朝后端发起 ajax 请求

并计算两个数字的和,最后结果展示在第3个input 框中.

1 创建项目-django_ajax

自行创建

2 添加static



3 urls.py

```
path('ajax_test/',views.ajax_test)
```

4 views.py

```
def ajax_test(reqeust):
    if reqeust.method == 'POST':
        d1 = reqeust.POST.get('d1')
        d2 = reqeust.POST.get('d2')
        #相加
        sum = int(d1) +int(d2)
        #返回结果
        return HttpResponse(sum)


    return render(reqeust,'ajaxtest.html')
```

5 ajaxtest.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
    {% load static %}


    <!-- 导入外部的css-->
    <link rel="stylesheet" href={% static 'bootstrap-3.4.1-
dist/css/bootstrap.min.css' %}>
    <!-- 导入外部的js-->
    <script src={% static "js/jquery-3.7.0.min.js" %}></script>
    <!-- 导入外部的js-->
    <script src={% static "bootstrap-3.4.1-dist/js/bootstrap.min.js" %}>
</script>

</head>
<body>
<input type="text" id="d1">+
<input type="text" id="d2">
<button id="btn">=</button>
计算结果：<span id="sum"></span>
</body>

<script>
    //给=按钮绑定点击事件
   $('#btn').click(function () {
       var d1 = $('#d1').val()
       var d2 = $('#d2').val()

       $.ajax({
           //1. 请求的地址 不写默认当前地址提交请求
           url:'',
           //2. 请求类型 不写默认为get请求
           type:'post',
           //3. data 传递的数据 格式: {}
           data:{
               d1:d1,
               d2:d2
           },
           //4. success: 成功回调函数 有结果时自动执行
```

```
            success:function(res){
                 //res是接收后端返回结果数据
                $('#sum').html(res)
            }


        })
    })

</script>
</html>
```

八、作业

8.1 完成本天的案例